

Audit serveur — détection de doublons et rapport d'aide au nettoyage d'espace disque

Audit serveur — détection de doublons et rapport d'aide au nettoyage d'espace disque

Document de travail personnel, hors périmètre du développement SILLON. Il documente un cas d'usage de SILLON en tant qu'outil d'analyse : exploiter un inventaire de fichiers serveur (CSV) pour produire un rapport à l'attention du service décisionnaire, en vue d'un nettoyage d'espace disque.

1. Fichier source

Généré sur le serveur audité par :

```
find . -type f -exec bash -c '
  for file; do
    hash=$(md5sum "$file" | awk "{print \$1}")
    name=$(basename "$file")
    taille=$(du -s "$file" | cut -f 1)
    echo "$name;$hash;$file;$taille"
  done
' _ {} + > resultat.csv
```

Un CSV **sans ligne d'en-tête**, séparateur `;`, avec exactement 4 colonnes, dans cet ordre :

Colonne	Contenu
1	Nom du fichier
2	Hash du fichier (empreinte de contenu, MD5)
3	Chemin complet (chemins Unix, séparateur <code>/</code>)
4	Taille en Ko

⚠ Cet ordre (chemin avant taille) est celui de la commande ci-dessus — différent d'un ordre nom/hash/taille/chemin qui aurait pu être supposé au premier abord. À respecter scrupuleusement à l'étape d'import (§2), sinon toutes les analyses qui suivent (requêtes SQL, script Python) chercheront la taille dans la colonne du chemin et inversement, sans erreur visible.

⚠ La taille (colonne 4) vient de `du -s`, qui renvoie par défaut des blocs de **1024 octets** — **déjà en Ko**, pas en octets. C'est l'espace réellement **alloué sur disque** (arrondi à la taille de bloc du filesystem, souvent 4 Ko), pas la taille logique exacte du fichier — pertinent ici puisque l'objectif est de chiffrer l'espace disque récupérable, mais à garder en tête si un total est comparé à une taille "apparente" (`ls -l`, `stat`) obtenue par ailleurs.

⚠ Séparateur `;` : si un nom de fichier ou un chemin contient un `;`, la ligne correspondante sera mal coupée à l'import. Avant d'importer un gros inventaire, vérifier par exemple avec `grep -c ';' resultat.csv` comparé au nombre de lignes total (`wc -l resultat.csv`) — un écart signale des noms de fichiers à problème à traiter au cas par cas.

2. Import dans SILLON

Onglet **Import** → dépose le CSV → SILLON détecte l'encodage/délimiteur et propose un type par colonne. Comme le fichier n'a pas d'en-tête, les colonnes sont nommées génériquement — les renommer et typer ainsi, **dans l'ordre réel du fichier** (§1) :

- colonne 1 → `nom_fichier`, Texte
- colonne 2 → hash, **Texte** (important : pas Entier, même si c'est un hex — sinon les zéros de tête sautent et les tris numériques faussent la comparaison)
- colonne 3 → `chemin_complet`, Texte
- colonne 4 → `taille_ko`, Entier

Valide : une table est créée dans la base personnelle. Son nom (visible ensuite dans l'onglet **Bases** → **Tables**) doit être reporté dans les requêtes SQL et le script ci-dessous, à la place de `<table>` / `NOM_TABLE`.

3. Requêtes SQL (détection de doublons uniquement)

Fichier : `sql/requetes_doublons.sql`, à exécuter dans l'onglet **Travaux**.

Deux requêtes :

1. **Résumé par groupe de hash** — nombre de copies et espace gaspillé (Ko) par groupe de fichiers identiques.
2. **Détail ligne par ligne** — chaque fichier en doublon, avec un statut original (le premier par chemin) ou doublon, pour préparer une liste de suppression.

Ces requêtes suffisent pour une exploration rapide dans SILLON sans dépôt de script. Le rapport complet (doublons + arborescence de dossiers + fichiers volumineux + PDF/Excel) nécessite le script Python ci-dessous.

4. Script Python (rapport complet)

Fichier : `scripts/rapport_nettoyage_disque.py`, à déposer dans l'onglet **Scripts**.

Contrat d'exécution SILLON : connexion base (`SILLON_DSN`) et répertoire de sortie (`SILLON_RESULTATS`) fournis automatiquement par variables d'environnement — rien à saisir dans le script. Bibliothèques utilisées : `psycopg2`, `pandas`, `matplotlib`, `openpyxl` (toutes disponibles dans l'image d'exécution SILLON).

Le script calcule :

1. **Doublons** — mêmes deux analyses que les requêtes SQL (résumé par hash + détail avec statut original/doublon).
2. **Taille par dossier et sous-dossier** — cumul récursif : un fichier dans `/a/b/c/` compte dans la taille de `/a`, `/a/b` et `/a/b/c`.
3. **Fichiers les plus volumineux** — top 50 par taille.

Et produit deux livrables dans le répertoire de résultats du job (récupérables ensuite depuis l'onglet **Suivi**) :

- `rapport_nettoyage_disque.pdf` — une page de synthèse graphique (top dossiers, top groupes de doublons, top fichiers, résumé chiffré), présentable directement au service décisionnaire.
- `rapport_nettoyage_disque.xlsx` — classeur de détail à 4 onglets (Synthèse, Doublons, Dossiers, Plus volumineux), pour que le service décisionnaire puisse filtrer/trier lui-même.

Avant exécution : ajuster `NOM_TABLE` (début du script) au nom réel de la table créée à l'import.

5. Script Python (variante : doublons filtrés par taille minimale)

Fichier : `scripts/rapport_doublons_taille_minimale.py`, à déposer dans l'onglet **Scripts**.

Version allégée du script du §4, pour répondre à un besoin différent : sur un inventaire complet, la plupart des groupes de doublons sont de petits fichiers (icônes, gabarits...) sans intérêt pour le nettoyage. Ce script ne s'intéresse qu'aux doublons dont récupérer l'espace vaut la peine — par exemple uniquement les fichiers de 30 Mo et plus, ou seulement 100 Mo et plus.

Deux différences avec le script du §4 :

- **Un seul livrable**, `rapport_doublons_taille_minimale.xlsx` (pas de PDF ni de graphique) — bibliothèques utilisées : `psycog2`, `pandas`, `openpyxl` (pas de `matplotlib`).
- **Filtre de taille minimale** : seuls les fichiers d'au moins `TAILLE_MIN_KO` (constante en tête de script, en Ko/Kio comme la colonne `taille_ko`) entrent dans l'analyse. Un fichier sous le seuil est exclu de l'inventaire analysé *avant* le calcul des doublons — un groupe de doublons dont l'original dépasse le seuil mais dont les copies sont plus petites peut donc apparaître avec moins de copies que dans le rapport complet du §4, voire disparaître si l'original se retrouve seul après filtrage.

Le classeur produit a 2 onglets :

- **Synthèse** — seuil utilisé, nombre de fichiers et volume analysés (après filtre), nombre de groupes de doublons, espace récupérable.
- **Doublons** — détail ligne par ligne (`hash`, `statut` original/doublon, `nom_fichier`, `chemin_complet`, `taille_mo`, `taille_ko`), groupes triés par espace récupérable décroissant (les doublons les plus rentables à supprimer en premier), original avant ses copies dans chaque groupe.

Avant exécution : ajuster `NOM_TABLE` et `TAILLE_MIN_KO` (début du script) — par exemple `TAILLE_MIN_KO = 30 * 1024` pour 30 Mo, `TAILLE_MIN_KO = 100 * 1024` pour 100 Mo.

6. Limites connues

- L'arborescence de dossiers suppose des chemins Unix absolus (`/...`) — non testé sur des chemins relatifs ou Windows.
- Le hash n'est pas recalculé par `SILLON` : sa fiabilité (détection de vrais doublons de contenu, pas de simples homonymes) dépend entièrement de l'algorithme et du calcul fait en amont, côté génération du CSV.
- Pas de gestion des liens symboliques ou des fichiers déjà supprimés entre la génération du CSV et l'exploitation du rapport — l'inventaire est une photo à un instant donné.